

پروژه طبقه‌بندی وسایل نقلیه ترافیکی

پارت دوم

گزارش ساختار جدید داده و آزمایش‌های ResNet18

مستندسازی تصمیم‌های مربوط به داده، Cascade، Transfer Learning، Error و Threshold
Analysis

مسعود ذوقیان

لینک پروژه : https://github.com/masoud-zoghian/Traffic_Vehicle_Classification

پاییز 1405

بوت کمپ هوش مصنوعی مکتب شریف

1) تغییر کامل ساختار داده‌ها

در این مرحله، ساختار کل داده‌های پروژه تغییر داده شد. به جای ادامه دادن با ساختار قبلی، داده‌های مورد نیاز این بخش در یک ساختار مستقل و مشخص با ریشه newData سازمان‌دهی شدند. هدف این تصمیم این بود که داده‌های آموزشی، داده‌های تست، داده‌های خارج از توزیع (Neysan/OOD) و فایل‌های مربوط به Split از یکدیگر جدا باشند و فرایند Cascade و ResNet18 روی یک ساختار قابل کنترل اجرا شود.

ساختار مشاهده‌شده در این مرحله به شکل زیر است:

- newData/
 - └─ dataset/
 - | └─ train/
 - | └─ clean/
 - └─ datasetv2_TrainClean/
 - | └─ train/
 - | └─ clean/
 - └─ dataTest/
 - | └─ test/
 - | └─ test_neysan_subset/
 - └─ neysan_ood/
 - └─ resnet_data_splits/

در کنار این ساختار داده، پوشه‌ی saved_models نیز برای ذخیره وزن مدل‌های آموزش‌دیده استفاده شد.

بخش	نقش در پروژه
dataset/train + clean	یکی از منابع داده آموزشی اصلی
datasetv2_TrainClean/train + clean	منبع دوم داده آموزشی
dataTest/test	مجموعه تست نهایی Phase 1؛ در آموزش وارد نشد
neysan_ood	داده خارج از کلاس‌های آموزش‌دیده برای بررسی OOD و Threshold
resnet_data_splits	نگهداری Manifest مربوط به Train/Validation و فهرست Phase 2
saved_models	ذخیره بهترین وزن‌های ResNet18 و Cascade و فایل Threshold

2) فایل — 1_data_audit / ممیزی و تصمیم‌گیری درباره داده

هدف این Notebook این بود که قبل از ساخت مدل، وضعیت واقعی داده‌ها بررسی شود و یک مجموعه داده کنترل‌شده برای آموزش ساخته شود. چهار منبع آموزشی بررسی شدند و مشخص شد هر چهار منبع دارای همان 8 کلاس هستند.

منبع	کلاس‌های موجود
newData/dataset/train	8 کلاس
newData/dataset/clean	8 کلاس
newData/datasetv2_TrainClean/train	8 کلاس
newData/datasetv2_TrainClean/clean	8 کلاس

کلاس‌های آموزشی در این مرحله عبارت بودند از ambulance، bus، kamyunet، kamyun، minibus، savari، taxi و vanet.

در این مرحله هنوز kamyun و kamyunet جدا بودند. ادغام این دو کلاس فقط برای مدل Cascade در مرحله دوم انجام شد؛ بنابراین داده اصلی و Split همچنان برچسب‌های 8 کلاسه را حفظ کردند.

2.1 - ساخت Inventory

- تابع build_inventory مسیر فایل، نام کلاس و پوشه منبع را برای تمام تصاویر معتبر جمع‌آوری کرد.
- پسوندهای معتبر شامل jpg، jpeg، png، bmp و webp بودند.
- برای هر تصویر سه اطلاعات اصلی ثبت شد: filepath، label و source_folder
- در مجموع 3313 تصویر از چهار منبع آموزشی جمع‌آوری شد.

کلاس	تعداد اولیه
savari	498
kamyun	493
taxi	484
autobus	464
kamyunet	456
minibus	394
ambulance	369
vanet	155
جمع	3313

2.2 - تصمیم مهم درباره Duplicate ها

برای پیدا کردن تصاویر تکراری، فقط نام فایل یا مسیر بررسی نشد. محتوای تصویر با PIL خوانده شد، به RGB تبدیل و به اندازه 64×64 تغییر داده شد و سپس MD5 روی بایت‌های تصویر نرمال‌شده محاسبه شد. بنابراین هدف، شناسایی تصاویر دارای محتوای یکسان حتی در صورت تفاوت‌های ظاهری در فایل/ذخیره‌سازی بود.

- 11 گروه Duplicate پیدا شد.
 - در مجموع 22 رکورد در این گروه‌ها قرار داشتند.
 - Duplicate ها بین dataset/train و dataset/clean دیده شدند.
 - در بررسی گروه‌ها Label Conflict مشاهده نشد؛ یعنی یک محتوای یکسان با دو برچسب متفاوت ثبت نشده بود.
- برای Deduplication تصمیم گرفته شد اگر یک گروه Duplicate وجود دارد، نسخه‌ای که مسیر آن شامل clean است نگه داشته شود و نسخه دیگر فقط از DataFrame حذف شود؛ فایل اصلی روی دیسک حذف نشد.

- 11 رکورد حذف منطقی شدند.
- تعداد داده از 3313 به 3302 تصویر یکتا رسید.
- توزیع نهایی پس از Deduplication : savari=498 ، kamyun=493 ، taxi=484 ، autobus=463 ، vanet=155 ، ambulance=361 ، minibus=392 ، kamyunet=456 .

2.3 - تصمیم درباره Train / Validation

- Split به صورت Stratified و بر اساس label اصلی 8 کلاسه انجام شد.
- VAL_RATIO برابر 0.20 انتخاب شد.
- SPLIT_SEED برابر 42 قرار گرفت.
- Split برای هر کلاس جداگانه انجام شد تا نسبت کلاس‌ها در Train و Validation حفظ شود.
- پس از Split ، Hash های Train و Validation با هم مقایسه شدند.
- Hash overlap برابر صفر شد؛ بنابراین بین Train و Validation نشت داده بر اساس این بررسی وجود نداشت.

کلاس	Train	Validation	نسبت Validation
ambulance	288	73	0.202
autobus	370	93	0.201
kamyun	394	99	0.201
kamyunet	364	92	0.202
minibus	313	79	0.202
savari	398	100	0.201
taxi	387	97	0.200
vanet	124	31	0.200
جمع	2638	664	≈20%

Manifest نهایی در مسیر newData/resnet_data_splits/inventory_train_val_split.csv ذخیره شد. این فایل مبنای مراحل بعدی قرار گرفت و در بخش Recovery نیز مستقیماً از روی آن داده‌ها دوباره بارگذاری شدند.

3 فایل — 2_3_dataset_transforms / آماده‌سازی Dataset و DataLoader

این Notebook با اجرای 1_data_audit.ipynb شروع می‌شود تا Inventory و Split قبلی دوباره ساخته/در دسترس قرار گیرند. سپس تنظیمات ورودی مدل و Dataset/Loader ها ساخته شدند.

3.1 – Mapping class اصلی

برای داده 8 کلاسه، class_to_idx به صورت مرتب و ثابت ساخته شد:

- ambulance=0 ، autobus=1 ، kamyun=2 ، kamyunet=3 ، minibus=4 ، savari=5 ، taxi=6 ، vanet=7

3.2 – Transform های Train و Validation

برای Train ، علاوه بر Resize و Normalize ، سه نوع تغییر تصادفی اعمال شد:

- RandomHorizontalFlip(p=0.5)
- RandomRotation با دامنه 10 درجه
- ColorJitter با brightness=0.2 ، contrast=0.2 و saturation=0.2

- سپس ToTensor و Normalize با مقادیر ImageNet انجام شد.
- برای Validation هیچ تغییر تصادفی اعمال نشد و فقط Resize < -- ToTensor < -- Normalize استفاده شد. این تصمیم باعث می‌شود ارزیابی روی Validation پایدار و بدون تغییرات تصادفی تصویر انجام شود.

VehicleDataset - 3.3

- کلاس VehicleDataset از torch.utils.data.Dataset ساخته شد.
- تصویر با PIL باز و به RGB تبدیل شد.
- Transform در زمان __getitem__ اعمال شد.
- نام کلاس با class_to_idx به عدد تبدیل شد.
- بعداً Dataset انعطاف‌پذیر شد تا بتواند از label_col های مختلف، از جمله coarse_label، استفاده کند.

DataLoader - 3.4

- Train Loader با batch_size=32 و shuffle=True ساخته شد.
- Validation Loader با batch_size=32 و shuffle=False ساخته شد.
- Batch نمونه دارای shape برابر [32, 3, 224, 224] و label shape برابر [32] بود.

4) تصمیم کلیدی برای معماری Cascade ساخت Coarse Model :

یکی از مهم‌ترین تصمیم‌های این نسخه از پروژه این بود که مدل مستقیماً 8 کلاس را به‌صورت یک شبکه واحد تشخیص ندهد. ابتدا یک مدل Coarse ساخته شد که 7 خروجی دارد و دو کلاس kamyun و kamyunet را در یک گروه به نام truck_merged ادغام می‌کند. سپس فقط زمانی که Coarse به truck_merged برسد، تصویر وارد مدل دوم یا Cascade می‌شود. این تصمیم عملاً مسئله را به دو مرحله تقسیم می‌کند: مرحله اول تشخیص گروه وسیله و مرحله دوم تفکیک دو کلاس مشابه kamyun و kamyunet

مدل	کلاس‌ها
Coarse	ambulance, autobus, minibus, savari, taxi, truck_merged, vanet
Cascade	kamyun, kamyunet

در نتیجه، تعداد داده Coarse همچنان 3302 تصویر است، اما تعداد کلاس‌ها از 8 به 7 کاهش می‌یابد؛ در مقابل Cascade فقط 949 تصویر شامل kamyun/kamyunet را دریافت می‌کند.

Cascade split	kamyun	kamyunet	جمع
Train	394	364	758
Validation	99	92	191
کل	493	456	949

5) Transfer Learning با ResNet18 — resnet_5_4

در این مرحله به‌جای CNN ساده، از ResNet18 از پیش آموزش‌دیده روی ImageNet استفاده شود. وزن‌های اولیه با torchvision و ResNet18_Weights.IMAGENET1K_V1 بارگذاری شدند.

- ResNet18 اصلی 11,689,512 پارامتر داشت و لایه fc اولیه 1000 خروجی داشت.
- لایه نهایی با یک Linear متناسب با تعداد کلاس های پروژه جایگزین شد.

5.1 Feature Extraction

در اولین آزمایش ResNet18، تمام پارامترهای شبکه فریز شدند و فقط لایه fc جدید آموزش داده شد.

ویژگی	مقدار
Trainable parameters	3,591
Total parameters	11,180,103
Trainable ratio	0.0321%
Epochs	10
Loss	CrossEntropyLoss
Optimizer	Adam
Learning Rate	0.001
Best Validation Accuracy	91.11% (Epoch 10)

بهترین وزن Feature Extraction در coarse_resnet18_feature_extraction.pt ذخیره شد.

5.2 Fine-Tuning کامل

در مرحله بعد تصمیم گرفته شد کل ResNet18 قابل آموزش باشد، اما برای جلوگیری از تغییر شدید وزن های ازپیش آموخته شده، Learning Rate برای Backbone و Head جدا شود.

بخش	Learning Rate	پارامتر
Backbone	0.0001	11,176,512
FC Head	0.001	3,591

- تمام 11,180,103 پارامتر در Fine-Tuning قابل آموزش شدند.
- 8 Epoch آموزش انجام شد.
- Best Validation Accuracy برابر 98.80% شد.
- بهترین مدل در coarse_resnet18_fine_tuning.pt ذخیره شد.

6 آزمایش Weighted Loss

با توجه به نامتوازن بودن کلاس ها، تصمیم گرفته شد یک آزمایش جداگانه با Class-Weighted Cross Entropy انجام شود. وزن هر کلاس بر اساس فرمول $\text{total_samples} / (\text{num_classes} \times \text{class_count})$ محاسبه شد؛ بنابراین کلاس های کم نمونه وزن بیشتری دریافت کردند.

Coarse Class	Train Count	Weight
ambulance	288	1.3085
autobus	370	1.0185
minibus	313	1.2040
savari	398	0.9469

taxi	387	0.9738
truck_merged	758	0.4972
vanet	124	3.0392

در این آزمایش یک مدل Fine-Tuning جدید از ابتدا ساخته شد تا وزن‌های آزمایش قبلی روی نتیجه اثر نگذارند .
نتیجه Best Validation Accuracy: برابر 99.10% در Epoch 7 بود و از Fine-Tuning معمولی 98.80% بهتر شد.

Metric	Weighted Loss
Accuracy	0.9910
Macro Precision	0.9866
Macro Recall	0.9911
Macro F1	0.9886

مدل در coarse_resnet18_fine_tuning_weighted.pt ذخیره شد و همین مدل بعداً برای Pipeline و Threshold استفاده شد.

(7) آزمایش Balanced Batch Sampler

در آزمایش دوم مربوط به عدم توازن، به جای تغییر Loss، نحوه نمونه برداری Train تغییر داده شد. برای هر کلاس وزن نمونه برابر $1 / \text{class_count}$ در نظر گرفته شد و WeightedRandomSampler با replacement=True ساخته شد.

- Sampler جایگزین shuffle=True در DataLoader شد.
- توزیع 10 Batch نمونه بررسی شد تا مشخص شود نمونه‌برداری به سمت تعادل حرکت کرده است.

کلاس	وزن نمونه‌برداری
truck_merged	0.0013193
savari	0.0025126
taxi	0.0025840
autobus	0.0027027
minibus	0.0031949
ambulance	0.0034722
vanet	0.0080645

نتیجه Best Validation Accuracy: برابر 98.80% شد؛ بنابراین Balanced Sampler در این تنظیمات نسبت به Weighted Loss بهبود بیشتری ایجاد نکرد.

Metric	Balanced Sampler
Accuracy	0.9880
Macro Precision	0.9859
Macro Recall	0.9826
Macro F1	0.9842

مدل در coarse_resnet18_fine_tuning_balanced.pt ذخیره شد.

8) ساخت و آموزش Cascade Model

پس از تصمیم Coarse ، داده‌های اصلی قبل از Merge فیلتر شدند تا فقط نمونه‌های kamyun و kamyunet برای مدل دوم باقی بمانند. بنابراین Cascade دقیقاً یک مسئله دوکلاسه است.

- Train 758 : 394 تصویر kamyun و 364 تصویر kamyunet
- Validation 191 : 99 تصویر kamyun و 92 تصویر kamyunet
- ResNet18 با Fine-Tuning کامل استفاده شد.
- Head LR=0.001 و Backbone LR=0.0001
- Loss = CrossEntropyLoss
- آموزش 8 Epoch

Best Validation Accuracy برابر 93.19% در Epoch 7 و 8 بود.

کلاس	Precision	Recall	F1	Support
kamyun	0.9886	0.8788	0.9305	99
kamyunet	0.8835	0.9891	0.9333	92
Macro Avg	0.9361	0.9340	0.9319	191

مدل در cascade_resnet18_kamyun_kamyunet.pt ذخیره شد.

9) ساخت Pipeline دو مرحله‌ای

تابع predict_vehicle خط لوله نهایی را پیاده‌سازی کرد. ابتدا تصویر وارد Coarse می‌شود. اگر خروجی Coarse یکی از کلاس‌های عادی باشد، همان خروجی نهایی است. اگر خروجی truck_merged باشد، تصویر به Cascade ارسال می‌شود تا بین kamyun و kamyunet تصمیم گرفته شود.

- برای Coarse از Softmax استفاده شد.
- کلاس با بیشترین احتمال انتخاب شد.
- در حالت غیر truck_merged ، confidence همان احتمال Coarse است.
- در حالت truck_merged ، confidence نهایی برابر حاصل ضرب confidence Coarse در confidence Cascade در نظر گرفته شد.
- نتیجه شامل coarse_prediction ، coarse_probs ، predicted_class ، confidence و در صورت نیاز cascade_probs بود.

یک Sanity Check روی 8 نمونه Validation از کلاس‌های مختلف انجام شد و همه نمونه‌های انتخاب‌شده درست پیش بینی شدند.

10) threshold_6 — طراحی threshold و needs_review

بعد از ساخت Pipeline ، تصمیم گرفته شد صرفاً یک predicted_class برگردانده نشود. هدف این بود که اگر مدل اطمینان کافی ندارد، نتیجه با یک پرچم needs_review علامت گذاری شود.

برای این کار، داده‌های neysan_ood به عنوان کلاس آموزش ندیده و خارج از توزیع استفاده شدند. این داده‌ها در آموزش مدل استفاده نشدند.

10.1 - تفکیک Neysan برای Calibration و Phase 2

- تعداد کل تصاویر neysan_ood برابر 554 بود.
 - داده‌ها با Seed ثابت به دو نیمه تقسیم شدند.
 - 277 تصویر برای Calibration Threshold استفاده شد.
 - 277 تصویر برای Phase 2 Test کنار گذاشته شد.
 - فهرست فایل‌های Phase 2 در neysan_phase2_test_files.csv ذخیره شد.
- این تصمیم مهم بود چون همان داده‌ای که برای انتخاب Threshold استفاده می‌شود نباید دوباره برای قضاوت نهایی عملکرد Threshold مصرف شود.

10.2 - مقایسه Confidence روی Neysan و Known

مجموعه	Mean Confidence	سایر شاخص‌ها
Known Validation	0.9876	p10=0.9899 ، min=0.3860 p25=0.9983
Neysan Calibration	0.9521	p75=0.9998 ، max=1.0000 p90=1.0000

این نتایج نشان دادند که Softmax Confidence به تنهایی برای تشخیص OOD بسیار دشوار است؛ زیرا Neysan نیز در بسیاری از موارد با confidence بسیار بالا پیش‌بینی شد. به همین دلیل Threshold باید با یک Trade-off مشخص انتخاب شود.

10.3 - Threshold Sweep

Threshold	Known FPR	Neysan TPR
0.500	0.30%	1.81%
0.600	0.90%	4.69%
0.700	1.81%	7.58%
0.800	2.11%	9.03%
0.900	2.26%	13.36%
0.950	3.92%	15.88%
0.970	5.12%	19.86%
0.990	10.09%	29.24%
0.995	15.51%	36.82%
0.999	29.67%	54.87%

در نهایت Threshold برابر 0.99 انتخاب شد. دلیل مستند این انتخاب، ایجاد یک نقطه میانی در Trade-off بود: در 0.99 حدود 29.24% از Neysan های Calibration زیر Threshold قرار گرفتند، در حالی که 10.09% از داده‌های Known نیز

به اشتباه Flag شدند. Threshold های بالاتر Neysan بیشتری را Flag می کردند، اما False Alarm روی داده های Known نیز به سرعت افزایش می یافت.

10.4 - ذخیره تصمیم Threshold

- CONFIDENCE_THRESHOLD = 0.99
- Known false positive rate ثبت شده 0.1009
- Neysan true positive rate ثبت شده 0.2924
- Known calibration set size = 664
- Neysan calibration set size = 277
- این اطلاعات در saved_models/confidence_threshold.json ذخیره شد.

10.5 - افزودن needs_review به Inference

تابع predict_vehicle توسعه داده شد تا confidence را با Threshold مقایسه کند. اگر confidence کمتر از 0.99 باشد، needs_review=True و در غیر این صورت False می شود.

در حالت Coarse معمولی، confidence بررسی می شود. در حالت Cascade، confidence نهایی حاصل ضرب دو مرحله بررسی می شود.

11) ارزیابی نهایی Phase 1 روی 8 کلاس شناخته شده

برای تست Phase 1، مجموعه dataTest/test استفاده شد و هیچ neysan در آن قرار نگرفت.

تعداد	کلاس
50	ambulance
50	autobus
50	kamyun
50	kamyunet
50	minibus
50	savari
50	taxi
15	vanet
365	جمع

کل Pipeline دو مرحله ای روی هر 365 تصویر اجرا شد و برای هر تصویر predicted_class، confidence و needs_review ثبت شد.

Metric	Phase 1
Accuracy	0.9781
Macro Precision	0.9671
Macro Recall	0.9800
Macro F1	0.9722
Weighted Precision	0.9799
Weighted Recall	0.9781

Weighted F1	0.9783
-------------	--------

Confusion Matrix نشان داد تنها 8 تصویر از 365 تصویر اشتباه طبقه‌بندی شدند؛ بنابراین Error Rate برابر 2.19% بود.

12) ارزیابی Phase 2 روی Neysan به‌عنوان کلاس دیده‌نشده

در Phase 2 ، 277 تصویر Neysan که در مرحله Threshold Calibration استفاده نشده بودند وارد Pipeline شدند . برچسب neysan فقط برای نگهداری سوابق ثبت شد و به‌عنوان Ground Truth برای یکی از کلاس‌های آموزش‌دیده، استفاده نشد.

نتیجه پیش‌بینی روی Neysan	تعداد
vanet	258
minibus	9
savari	9
autobus	1
جمع	277

Metric	مقدار
Mean confidence	0.9538
Neysan needs_review rate	27.44%
Known Phase 1 needs_review rate	13.42%
Difference	14.01 percentage points

این نتیجه نشان می‌دهد Threshold توانسته روی Neysan نرخ Flag شدن بیشتری نسبت به داده‌های Known ایجاد کند، اما بسیاری از تصاویر Neysan همچنان با confidence بالا توسط مدل به یکی از کلاس‌های شناخته‌شده نسبت داده شدند . بنابراین needs_review یک سازوکار کمکی برای شناسایی موارد مشکوک است، نه یک OOD detector کامل.

13) Error Analysis

برای تحلیل خطا، تمام تصاویر اشتباه Phase 1 جدا شدند و علاوه بر true label و predicted class ، confidence و needs_review نیز بررسی شد.

True	Prediction	تعداد خطا
ambulance	vanet	2
kamyun	kamyunet	2
ambulance	savari	1
kamyun	minibus	1
taxi	savari	1
taxi	vanet	1

از 8 خطای Phase 1 ، برخی خطاها confidence پایین داشتند و با needs_review=True مشخص شدند، اما دو خطای مهم با confidence بالا رخ دادند: یک ambulance با confidence=0.999620 و یک kamyun با confidence=0.999821 به kamyunet نسبت داده شد. این موضوع نشان می‌دهد confidence بالا الزاماً به معنی پیش بینی صحیح نیست.

تصاویر تمام خطاها نیز به صورت Visual Inspection در Notebook نمایش داده شدند تا امکان بررسی ظاهری نمونه‌های اشتباه وجود داشته باشد.

14) جمع‌بندی نتایج مدل‌ها

نتیجه	Best Validation Accuracy	آزمایش
بهبود قابل توجه نسبت به شروع Transfer Learning	91.11%	ResNet18 Feature Extraction (Coarse)
بهبود شدید نسبت به Feature Extraction	98.80%	ResNet18 Fine-Tuning (Coarse)
بهترین Validation در آزمایش‌های Coarse	99.10%	Weighted Loss (Coarse)
مشابه Fine-Tuning معمولی	98.80%	Balanced Batch Sampler (Coarse)
تفکیک kamyun و kamyunet	93.19%	Cascade ResNet18
8 خطا از 365 تصویر	97.81% Accuracy	Full Phase 1 Cascade Test

15) خلاصه ساختار پوشه‌ها و فایل‌های کلیدی

کار انجام‌شده	پوشه / فایل
منبع آموزشی 8 کلاس	newData/dataset/train
منبع آموزشی 8 کلاس؛ در Duplicate ترجیح داده شد	newData/dataset/clean
منبع آموزشی دوم	newData/datasetv2_TrainClean/train
منبع آموزشی دوم	newData/datasetv2_TrainClean/clean
تست 1 Phase ، جدا از آموزش	newData/dataTest/test
داده OOD برای Calibration و 2 Phase	newData/neysan_ood
Manifest نهایی Train/Validation	newData/resnet_data_splits/inventory_train_val_split.csv
فهرست 277 تصویر Neysan برای 2 Phase	newData/resnet_data_splits/neysan_phase2_test_files.csv
بهترین Coarse Feature Extraction	saved_models/coarse_resnet18_feature_extraction.pt
بهترین Coarse Fine-Tuning	saved_models/coarse_resnet18_fine_tuning.pt
بهترین Coarse Weighted Loss ؛ مدل استفاده‌شده در Pipeline	saved_models/coarse_resnet18_fine_tuning_weighted.pt
مدل Coarse با Balanced Sampler	saved_models/coarse_resnet18_fine_tuning_balanced.pt
مدل Cascade دو کلاس	saved_models/cascade_resnet18_kamyun_kamyunet.pt
Threshold نهایی 0.99 و آمار Calibration	saved_models/confidence_threshold.json

16) مهم‌ترین تصمیم‌های مهندسی و داده‌ای این مرحله

- کل ساختار داده برای این بخش از پروژه باز طراحی و در newData متمرکز شد.
- داده‌های Train/Validation از چهار منبع آموزشی جداگانه جمع‌آوری و در یک Inventory واحد ادغام شدند.
- داده Test در dataTest/test جدا نگه داشته شد و در آموزش استفاده نشد.
- Neysan به‌عنوان کلاس آموزش‌نندیده در neysan_ood نگه داشته شد و وارد کلاس‌های آموزش اصلی نشد.
- Duplicate ها قبل از Split شناسایی شدند تا نشت داده ایجاد نشود.
- Duplicate ها نسخه clean ترجیح داده شد، ولی فایل‌های فیزیکی حذف نشدند.
- Split به صورت Stratified با Seed=42 و نسبت Validation 20% انجام شد.
- برای Coarse ، kamyun و kamyunet به truck_merged ادغام شدند، ولی داده اصلی 8 کلاس حفظ شد.
- برای تفکیک kamyun و kamyunet یک Cascade جدا ساخته شد.
- ResNet18 با وزن‌های ImageNet به‌عنوان مدل Transfer Learning انتخاب شد.
- ابتدا Feature Extraction و سپس Fine-Tuning انجام شد.
- برای Fine-Tuning ، Learning Rate برای Backbone و Head جدا انتخاب شد.
- برای عدم توازن کلاس‌ها دو راهکار مستقل آزمایش شد: Weighted Loss و Balanced Sampling
- Weighted Loss بهترین Validation Accuracy Coarse را به 99.10% رساند.
- Threshold با داده Neysan Calibration انتخاب شد، اما Neysan Phase 2 جدا نگه داشته شد تا ارزیابی نهایی مستقل باشد.
- Threshold نهایی 0.99 شد و در inference با needs_review استفاده شد.
- در نهایت علاوه بر Accuracy، Precision، Recall، F1، Confusion Matrix، Confidence و Error Analysis نیز بررسی شدند.